

popcount_am.sv — Nearest-Prototype Associative Memory + Co-Simulation Flow

This note documents the third verified RTL block of the 1024-HDC core: the **associative memory (AM)** that turns a query hypervector into a class label. It covers what the block computes, how the SystemVerilog implements it, and how the automated co-simulation proves it bit-for-bit against the Python golden reference.

1. What the associative memory does

After encoding, an EMG window becomes a single 1024-bit **query** hypervector. Training produced one **prototype** hypervector per gesture class (the majority bundle of all training windows of that class). Classification is then a nearest-neighbour search in Hamming space:

For each class `k`, count how many bits differ between the query and the prototype (optionally ignoring pruned bit positions via a mask), then pick the class with the **smallest** distance.

Formally, matching `hdc_ref.HDCEngine.classify`:

```
dist[k] = popcount( (query XOR proto[k]) AND mask )
best    = argmin_k dist[k]      # first index wins on a tie (NumPy argmin)
```

Two details matter for bit-exactness:

- **Masking happens before the popcount.** The pruning mask (Hook A / Twist 1/2) zeroes bit positions that should not contribute to the distance. `mask = all ones` is the unmasked case.
- **Tie-break is "first index wins."** NumPy's `argmin` returns the *lowest* index among equal minima. The RTL reproduces this with a strict `<` comparison while scanning classes in ascending order.

2. The RTL: rtl/popcount_am.sv

Interface

Signal	Dir	Width	Meaning
<code>clk</code> , <code>rst_n</code>	in	1	clock / active-low async reset
<code>proto_we</code> , <code>load_idx</code> , <code>load_vec</code>	in	1 / <code>IDX_W</code> / <code>D</code>	prototype write port (store <code>proto[load_idx] = load_vec</code>)
<code>mask_we</code> , <code>mask_vec</code>	in	1 / <code>D</code>	pruning-mask write port (defaults to all-ones after reset)

Signal	Dir	Width	Meaning
<code>q_valid</code> , <code>query_vec</code>	in	1 / <code>D</code>	classify request
<code>out_valid</code>	out	1	pulses one clock after <code>q_valid</code>
<code>best_idx</code>	out	<code>IDX_W</code>	index of the nearest prototype
<code>best_dist</code>	out	<code>DIST_W</code>	its masked Hamming distance

Parameters: `WORDS=16` , `BITS_PER_WORD=64` (`D=1024`) , `N_CLASS=8` , `IDX_W=$clog2(N_CLASS)` , `DIST_W=$clog2(D+1)=11` (distance range 0...1024).

Operation

1. **Storage.** `proto[0:N_CLASS-1]` and `mask` are registers loaded through the write ports. The mask resets to all-ones so an unloaded AM behaves as plain (unmasked) Hamming search.
2. **Distance + argmin (combinational).** A `popcount()` function sums the bits of `(query ^ proto[k]) & mask` (synthesises to an adder tree). A single `always_comb` loop scans all `N_CLASS` classes, keeping the running minimum with a **strict** `<` so the lowest index survives a tie. `best_dist_c` is initialised to all-ones (`> D`) so class 0 always sets the first candidate.
3. **Registered decision.** On `q_valid` , `best_idx` / `best_dist` are latched and `out_valid` is asserted on the next clock — a clean one-cycle handshake.

The classifier latency is one cycle; the heavy work (eight 1024-bit popcounts) is combinational, which is fine for simulation and synthesises to a modest, low-`f` adder-tree cloud that can later be pipelined if timing requires.

3. Verification methodology — "Python is truth"

Every RTL block in this project is checked the same way: a Python golden emits random stimulus + expected outputs as flat `$readmemh` files; a SystemVerilog testbench replays them through the DUT and compares bit-for-bit. There is **no hand-written SystemVerilog golden** in the loop, so the reference cannot silently drift from the model used for the accuracy results.

Golden vectors — `python_ref/generate_vectors.py --am`

`hdc_ref.export_am_cosim` writes, for `count` cases:

File	Lines	Contents
<code>am_proto.hex</code>	<code>count · N_CLASS · WORDS</code>	all prototypes, class-major (16 words each)
<code>am_mask.hex</code>	<code>count · WORDS</code>	per-bit pruning mask
<code>am_query.hex</code>	<code>count · WORDS</code>	query hypervector
<code>am_expect.hex</code>	<code>count</code>	<code>(best_idx << 16) best_dist</code> per case
<code>meta.txt</code>	—	<code>count, D, words, n_class, seed</code>

Stimulus coverage is deliberately adversarial for the corner cases:

- ~20 % of cases use an **all-ones mask** (unmasked Hamming) and the rest use a random mask, so the masked popcount path is exercised heavily.
- ~12 % of cases **duplicate one prototype onto another**, forcing two classes to the *exact* same minimum distance. The golden then returns the lower index; the RTL must agree — this is the only way to prove the tie-break logic.

Testbench — `tb/tb_am_cosim.sv`

Per case the TB: loads all `N_CLASS` prototypes (one per clock through the write port), loads the mask, then pulses `q_valid` with the query and waits one clock for `out_valid`. It compares both `best_idx` and `best_dist` against `am_expect.hex`. Any mismatch prints the case number with expected-vs-got and ends the run with `$fatal` (non-zero sim exit); a clean sweep ends with `$finish`.

Harness — `sim/run_am_cosim.do`

```
vsim -c -do sim/run_am_cosim.do
```

One command: regenerate vectors → build a fresh `work` lib → compile `popcount_am.sv` + TB → simulate. Override the case count with the `NUM_CASES` environment variable (default 500).

Co-simulation flow

<code>generate_vectors.py --am</code>	<code>tb_am_cosim.sv</code>	<code>popcount_am.sv</code>
-----	-----	-----
<code>random protos / mask / query</code>	<code>-> \$readmemh am_*.hex</code>	
<code>hdc_ref.classify (golden)</code>	<code>-> am_expect.hex (idx,dist)</code>	
	<code>load protos + mask</code>	<code>-> store registers</code>
	<code>drive query, q_valid</code>	<code>-> popcount + argmin</code>
	<code>compare idx & dist</code>	<code><- best_idx / best_dist</code>
	<code>PASS (\$finish) / FAIL (\$fatal)</code>	

4. Result

```
tb_am_cosim: nearest-prototype AM vs Python golden
VECDIR  = python_ref/vectors/cosim_am
CASES   = 500    (D=1024, N_CLASS=8)
PASS: all 500 AM cases match the Python golden bit-for-bit.
```

500 / 500 cases pass on ModelSim SE-64 10.6e, including the forced-tie and all-ones-mask cases.

`popcount_am.sv`, `tb_am_cosim.sv`, and the four `am_*.hex` vector files are added to the ModelSim project (`1024HDC.mpf`) for GUI use.

This completes the third of the core HDC datapath blocks (XOR-bind+permute → majority bundle → nearest-prototype AM), all verified by the same Python-is-truth co-simulation methodology.